

Ramaiah Institute of Technology
(Autonomous Institute, Affiliated to VTU)
Department of Computer Science and Engineering

Course Name: Object Oriented Programming Course

Code: CS34

Credits: 3:0:0

Term: October – January 2022

Faculty:
Mamatha A

Ramaiah Institute of Technology
(Autonomous Institute, Affiliated to VTU)
Department of Computer Science and Engineering

UNIT IV

EXCEPTION HANDLING & MULTITHREADED PROGRAMMING

AGENDA

Exception Handling:

- Introduction -Errors and Its Types
- Introduction -Exception
- Exception Handling Fundamentals
- Exception Types
- Uncaught Exceptions
- Using try and catch

ERRORS & TYPES

- An **error** is an **illegal statement or operation** performed by the programmer that causes the application or the program to behave abnormally and restricts the application from performing its tasks efficiently.
- **Errors** usually remain undetected until the program is **compiled or executed**.
- Some may cause the program's termination or may inhibit the program from getting compiled or executed.

ERRORS & TYPES

Types of Errors in JAVA.

□ Syntax Errors(Compile Time Errors)

□ Runtime Errors

□ Logical Errors

EXCEPTION

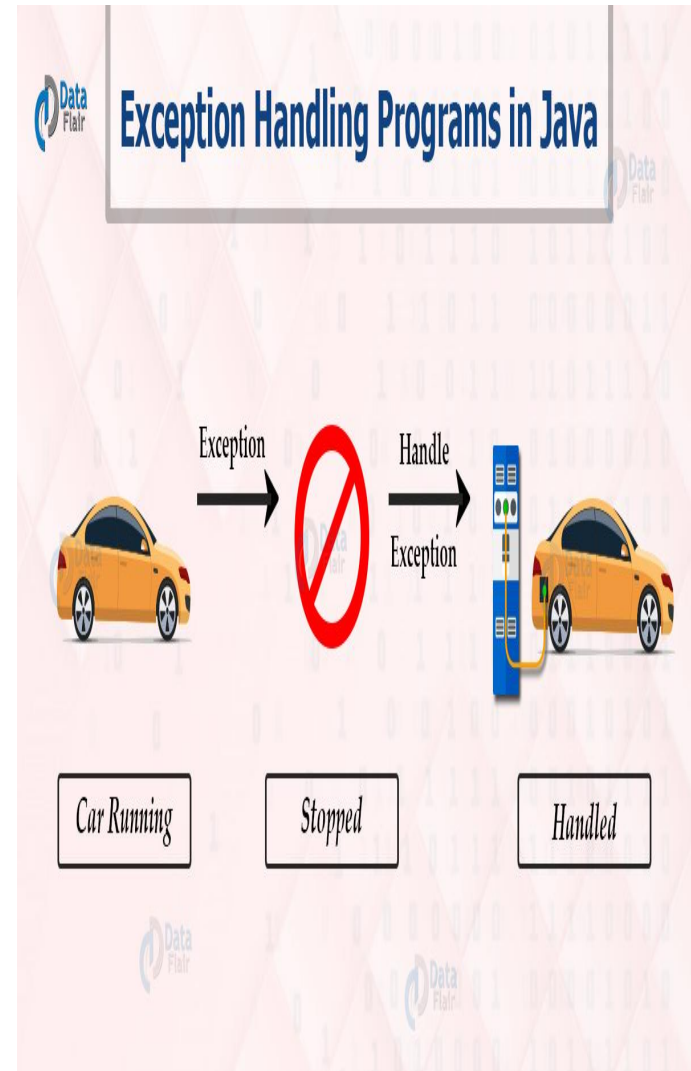
■ An **exception** is an **abnormal condition** that arises in a code sequence at **run time** and disrupts the normal flow of instructions.

■ In other words, an exception is a **run-time error**.

■ In computer languages that do not support exception handling, errors must be checked and handled manually—typically through the use of **error codes**.

■ This approach is as cumbersome as it is troublesome.

■ **Java's exception handling** avoids these problems and, in the process, brings run-time error management into the object oriented world.



EXCEPTION HANDLING FUNDAMENTALS

Java exception handling is managed using five keywords:

- **try:** A suspected code segment that needs to be monitored for exceptions is kept inside **try** block.
- **catch:** The remedy code is written within **catch** block.
- **throw:** System generated exceptions are automatically thrown by the Java run-time system. To manually throw an exception , the keyword **throw** is used
- **throws:** If a method cannot handle any exception by its own and some subsequent methods needs to handle them, then a method can be specified with **throws** keyword with its declaration.
- **finally:** block should contain the code to be executed after finishing try-block

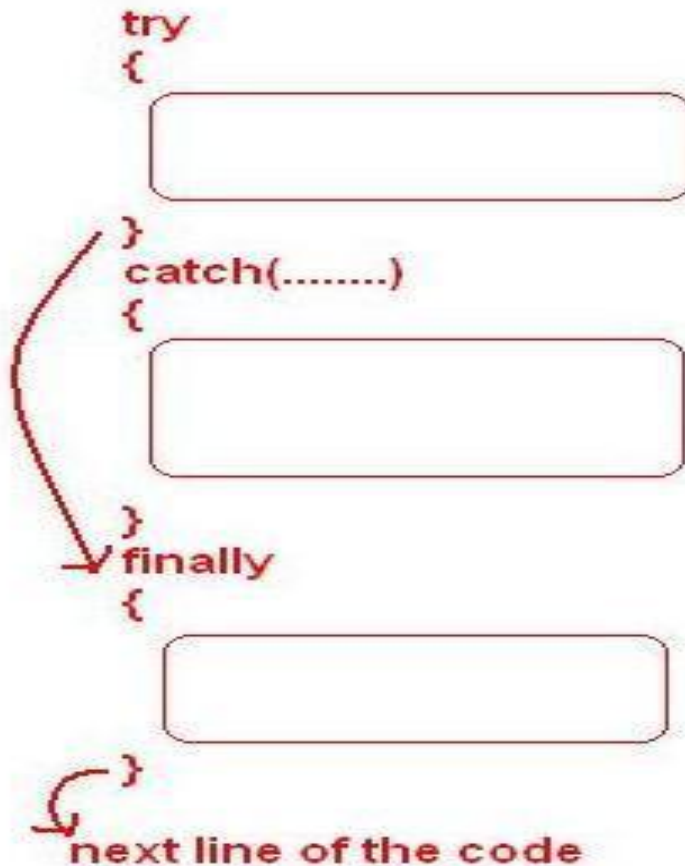
Exception handling fundamentals

This is the general form of an exception-handling block:

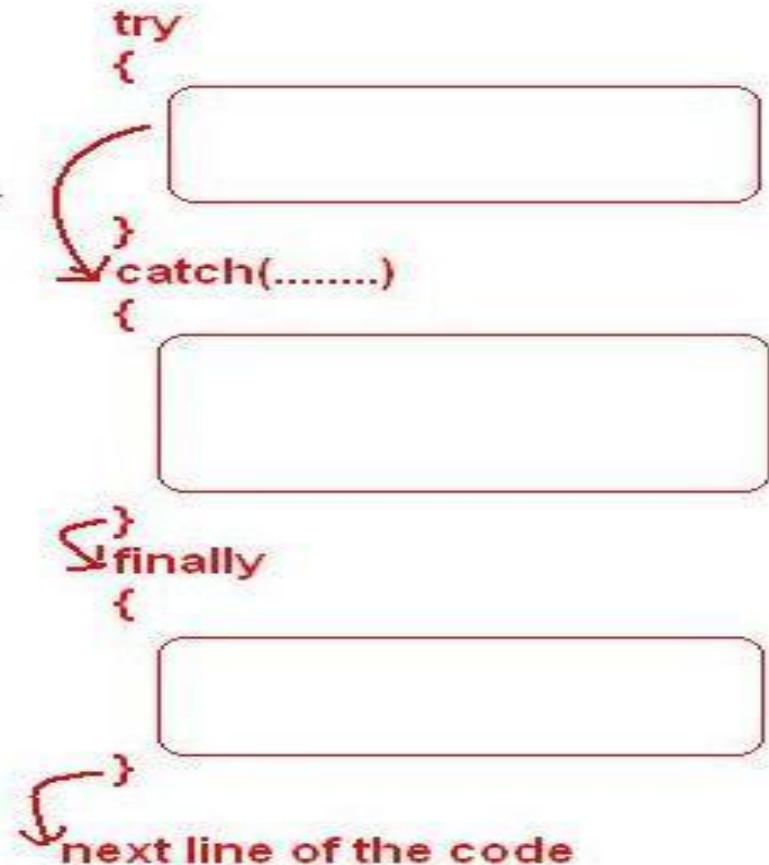
```
try {  
    // block of code to monitor for errors  
}  
catch (ExceptionType1 exOb) {  
    // exception handler for ExceptionType1  
}  
catch (ExceptionType2 exOb) {  
    // exception handler for ExceptionType2  
}  
// ...  
finally {  
    // block of code to be executed after try block ends  
}
```


EXCEPTION HANDLING FUNDAMENTALS

No exceptions thrown:



An exception arises :



EXCEPTION TYPES

- All the exception types are the derived classes of built-in class, **Throwable**.
- It has two subclasses, **Exception** and **Error**.
- **Exception** class is used for exceptional conditions that user programs should catch.
- We can inherit from this class to create our own custom exception types.

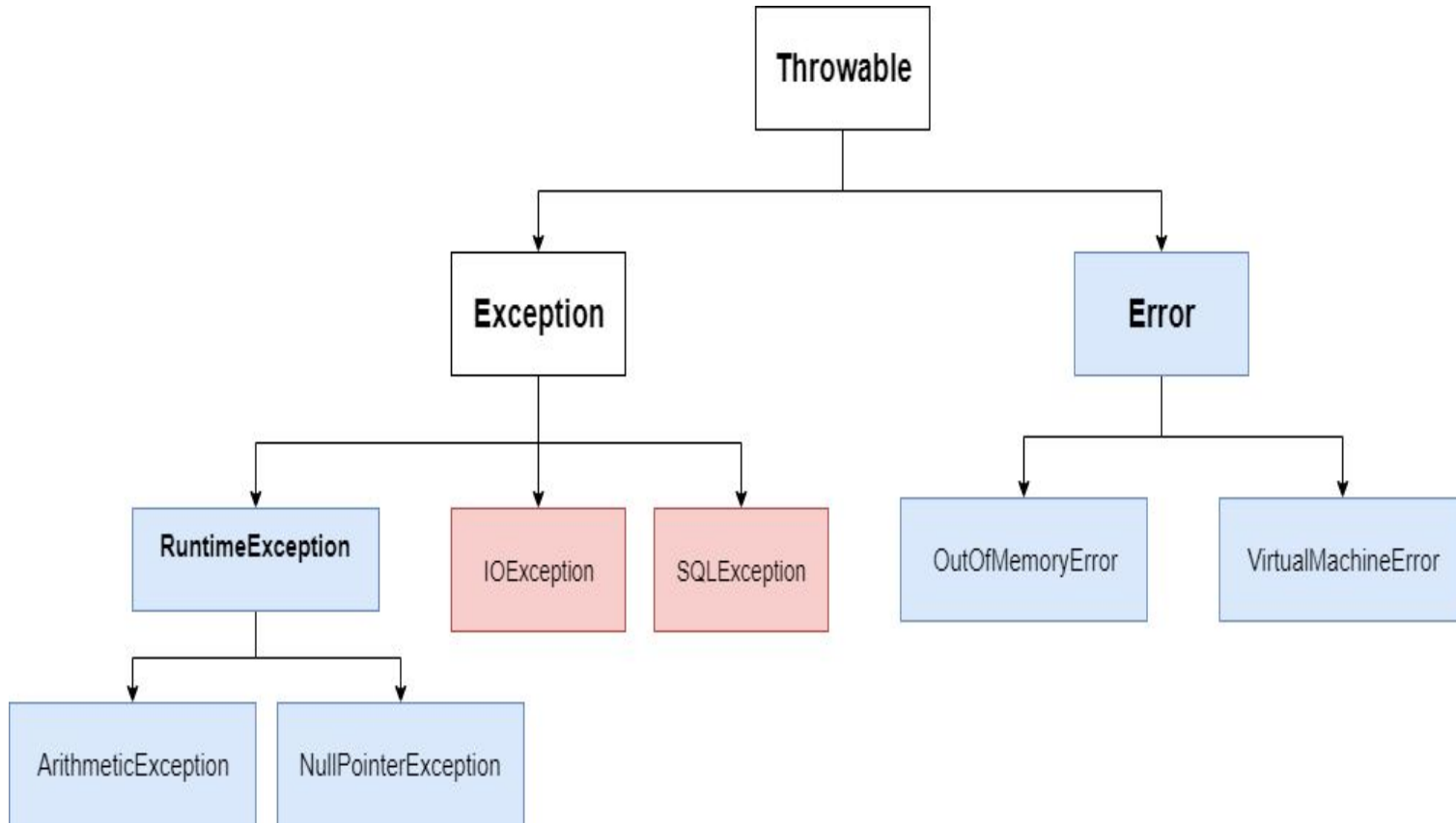
EXCEPTION TYPES

- There is an important subclass of Exception, called **RuntimeException**.
- Exceptions of this type are automatically defined for the programs that we write and include things such as division by zero and invalid array indexing.

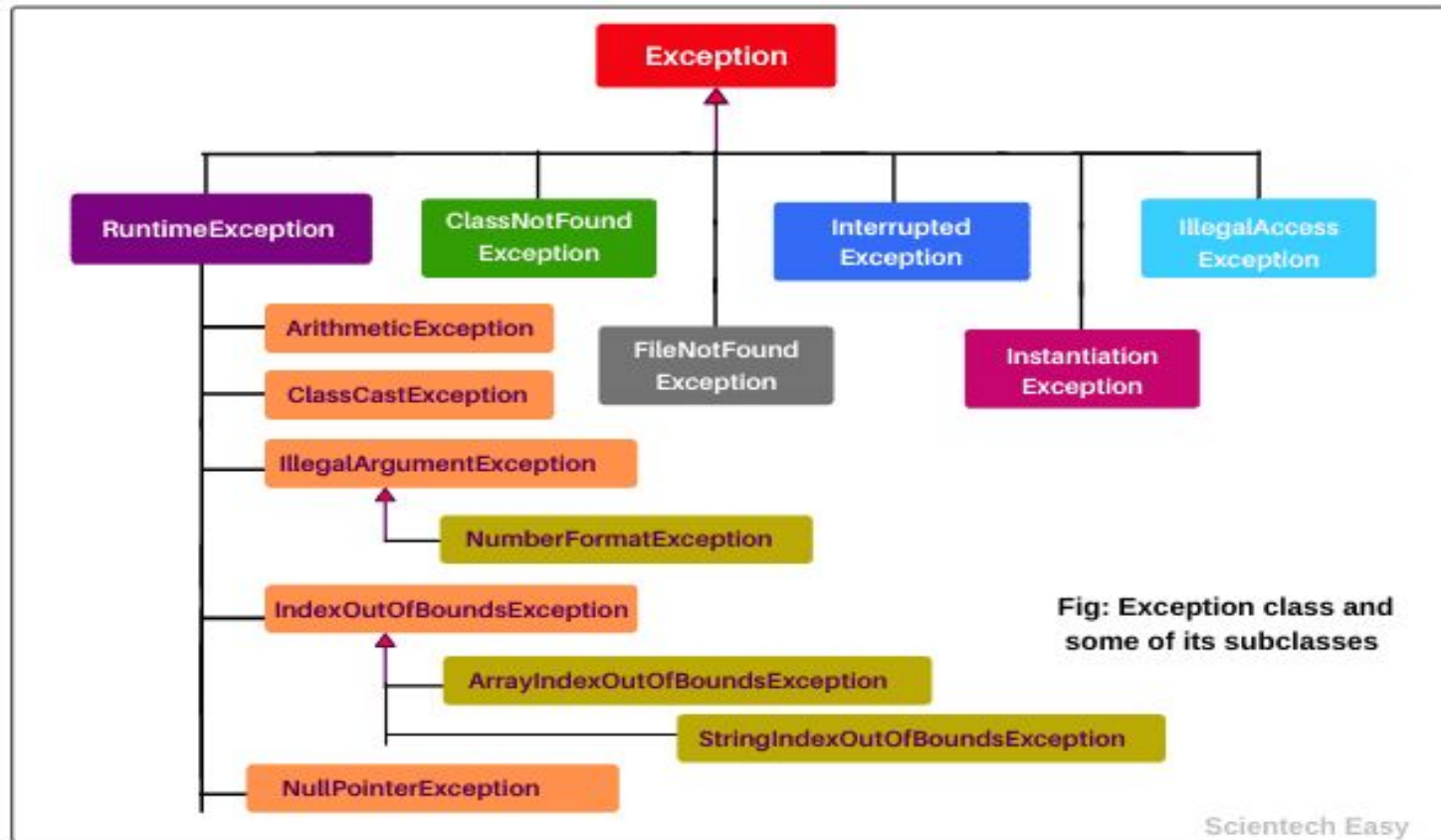
EXCEPTION TYPES

- ❑ **Error** class defines exceptions that are not expected to be caught under normal circumstances by program.
- ❑ Exceptions of type Error are used by the Java run-time system to indicate errors with the run-time environment.
- ❑ Stack overflow/ Out of Memory is an example of such an error.

Exception class hierarchy



Exception class hierarchy



Uncaught exceptions

```
class Exc0  
{  
    public static void main(String[] args)  
    {  
        int d=0;  
        int a= 42/d;  
        System.out.println(“End of the Program”);  
    }  
}
```

Ramaiah Institute of Technology
(Autonomous Institute, Affiliated to VTU)
Department of Computer Science and Engineering

Course Name: Object Oriented Programming Course

Code: CS34

Credits: 3:0:0

Term: October – January 2022

Faculty:
Mamatha A

Ramaiah Institute of Technology
(Autonomous Institute, Affiliated to VTU)
Department of Computer Science and Engineering

UNIT IV

EXCEPTION HANDLING & MULTITHREADED PROGRAMMING

RECAP...

Exception Handling:

- Introduction -Errors and Its Types
- Introduction -Exception
- Exception Handling Fundamentals
- Exception Types
- Uncaught Exceptions
- Using try and catch

AGENDA

Exception Handling:

- Multiple Catch Clauses
- Usage of keyword – **throw**
- Usage of keyword – **throws**

MULTIPLE CATCH CLAUSES

- A **try** block can be followed by **one or more catch blocks**.
- In some programs, more than one exception could be raised by a single piece of code.
- To handle this type of situation, we can specify two or more catch clauses, each catching a different type of exception.
- When an exception is thrown, each catch statement is inspected in order, and the first one whose type matches that of the exception is executed.
- After one catch statement executes, the others are bypassed, and execution continues after the try/catch block.

MULTIPLE CATCH CLAUSES

- While using multiple catch blocks, we should give the exception types in a hierarchy of subclass to superclass.
- Because, catch statement that uses a superclass will catch all exceptions of its own type plus all that of its subclasses.
- Hence, the subclass exception given after superclass exception is never caught and is a unreachable code, that is an error in Java.

MULTIPLE CATCH CLAUSES

```
main()
{
    try
    {

    }
    catch(Exception e)
    {

    }
    catch(RuntimeException e)
    {

    }
}
```

```
main()
{
    try
    {

    }
    catch(RuntimeException e)
    {

    }
    catch(Exception e)
    {

    }
}
```

THROW KEYWORD

- We have seen catching the exceptions that are thrown by the Java run-time system.
- It is possible for our own program to throw an exception explicitly, using the throw statement.

THROW KEYWORD

□ The general form of throw is shown here:

throw ThrowableInstance;

□ Here, ThrowableInstance must be an **object of type Throwable or a subclass of Throwable.**

□ There are two ways we can obtain a Throwable object:

- using a **parameter** in a **catch clause**, or
- creating one with the **new operator**.

Ramaiah Institute of Technology
(Autonomous Institute, Affiliated to VTU)
Department of Computer Science and Engineering

Course Name: Object Oriented Programming Course

Code: CS34

Credits: 3:0:0

Term: October – January 2022

Faculty:
Mamatha A

Ramaiah Institute of Technology
(Autonomous Institute, Affiliated to VTU)
Department of Computer Science and Engineering

UNIT IV

EXCEPTION HANDLING & MULTITHREADED PROGRAMMING

RECAP...

Exception Handling:

- Multiple Catch Clauses
- Usage of keyword – **throw**

AGENDA

Exception Handling:

- Usage of keyword – **throws**
- Usage of keyword - **finally**

THROWS KEYWORD

- If a method is capable of causing an exception that it does not handle, it must specify this behavior so that callers of the method can guard themselves against that exception.
- You do this by including a throws clause in the method's declaration.
- A throws clause lists the types of exceptions that a method might throw.

THROWS KEYWORD

- This is necessary for all exceptions, except those of **type Error or RuntimeException, or any of their subclasses.**
- All other exceptions that a method can throw must be declared in the throws clause.
- If they are not, a compile-time error will result.

THROWS KEYWORD

- The general form of a method declaration that includes a throws clause:

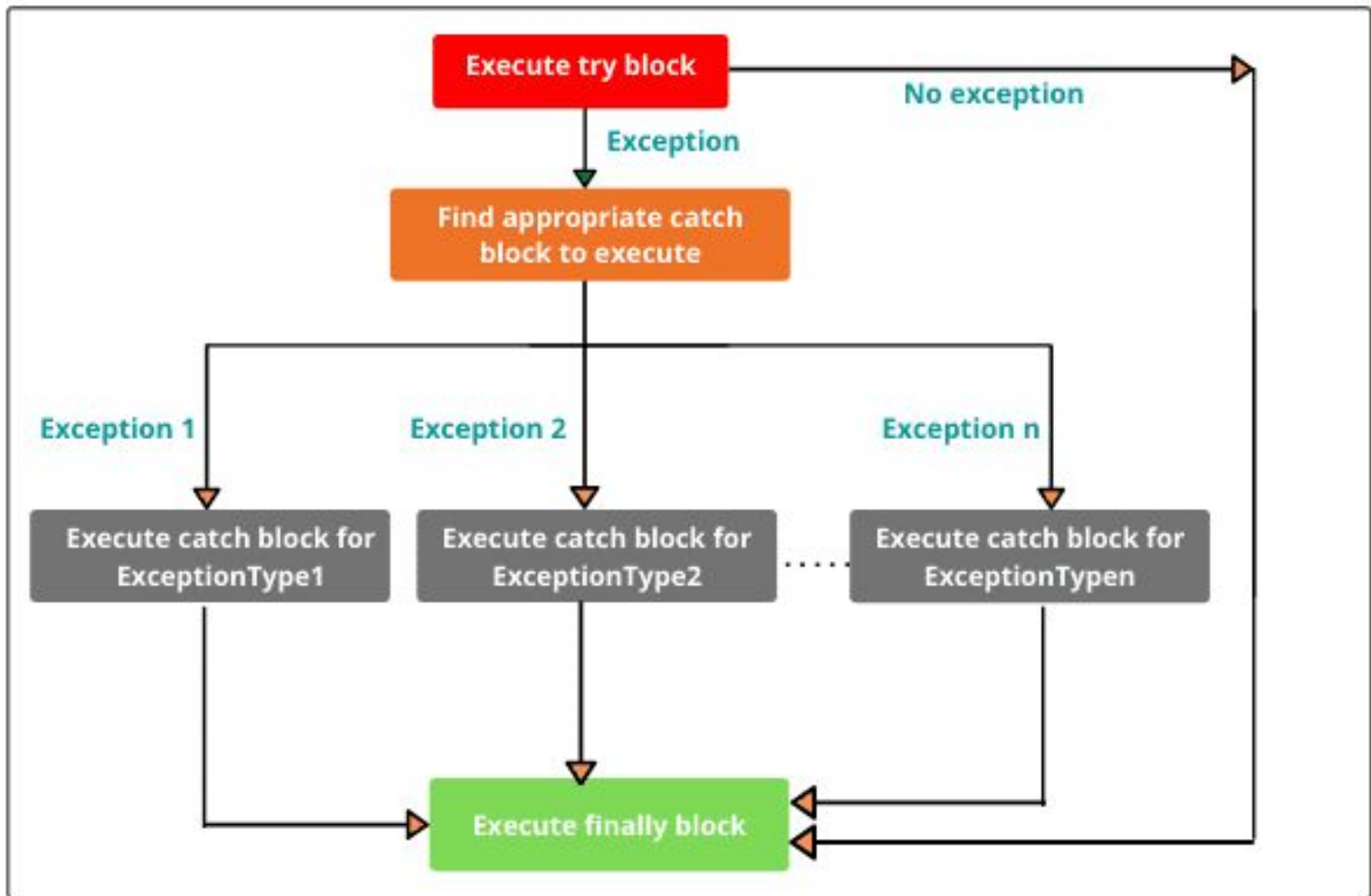
```
type method-name(parameter-list) throws exception-list  
{  
    // body of method  
}
```

- Here, exception-list is a comma-separated list of the exceptions that a method can throw.

FINALLY KEYWORD

- A “**finally**” is a keyword used to create a block of code that follows a **try or catch block**.
- A finally block contains all the crucial codes such as closing connections, stream, etc that is always executed whether **an exception occurs within a try block or not**.
- If an exception is thrown but not handled by any catch block, finally block executes
- finally block will be executed after a try/catch block has completed and before the code following the try/catch block

FINALLY KEYWORD



FINALLY KEYWORD

□ Any time a method is about to return to the caller from **inside a try/catch block**, via

- an uncaught exception
- an explicit return statement

the **finally clause** is executed just before the method returns

Ramaiah Institute of Technology
(Autonomous Institute, Affiliated to VTU)
Department of Computer Science and Engineering

Course Name: Object Oriented Programming Course

Code: CS34

Credits: 3:0:0

Term: October – January 2022

Faculty:
Mamatha A

**Ramaiah Institute of Technology
(Autonomous Institute, Affiliated to VTU)
Department of Computer Science and Engineering**

UNIT IV

EXCEPTION HANDLING & MULTITHREADED PROGRAMMING

AGENDA

Multithreaded Programming:

- Introduction to MultiThreading
- Java Thread Model
- The Main Thread

INTRODUCTION TO MULTITHREADING

- Java provides built-in support for **multithreaded programming**.
- A **multithreaded program** contains two or more parts that can run concurrently.
- Each part of such a program is called a **thread**, and each thread defines a separate path of execution.
- Thus, multithreading is a specialized form of **multitasking**.
- There are two distinct types of multitasking: **process-based** and **thread-based**.

INTRODUCTION TO MULTITHREADING

Process-based multitasking :

]A process is a program that is executing.

]Process-based multitasking is the feature that allows your computer to run two or more program concurrently.

]For example, process-based multitasking enables you to run the Java compiler at the same time you are using a text editor.

INTRODUCTION TO MULTITHREADING

Process-based multitasking :

- In process-based multitasking, a program is the smallest unit of code that can be dispatched by the scheduler.
- Process-based multitasking requires more overhead.
- The process to Process communication is expensive.
- Processes are heavyweight tasks that require their own separate address spaces.
- Context switching from one process to another is also costly.

INTRODUCTION TO MULTITHREADING

Thread-based multitasking :

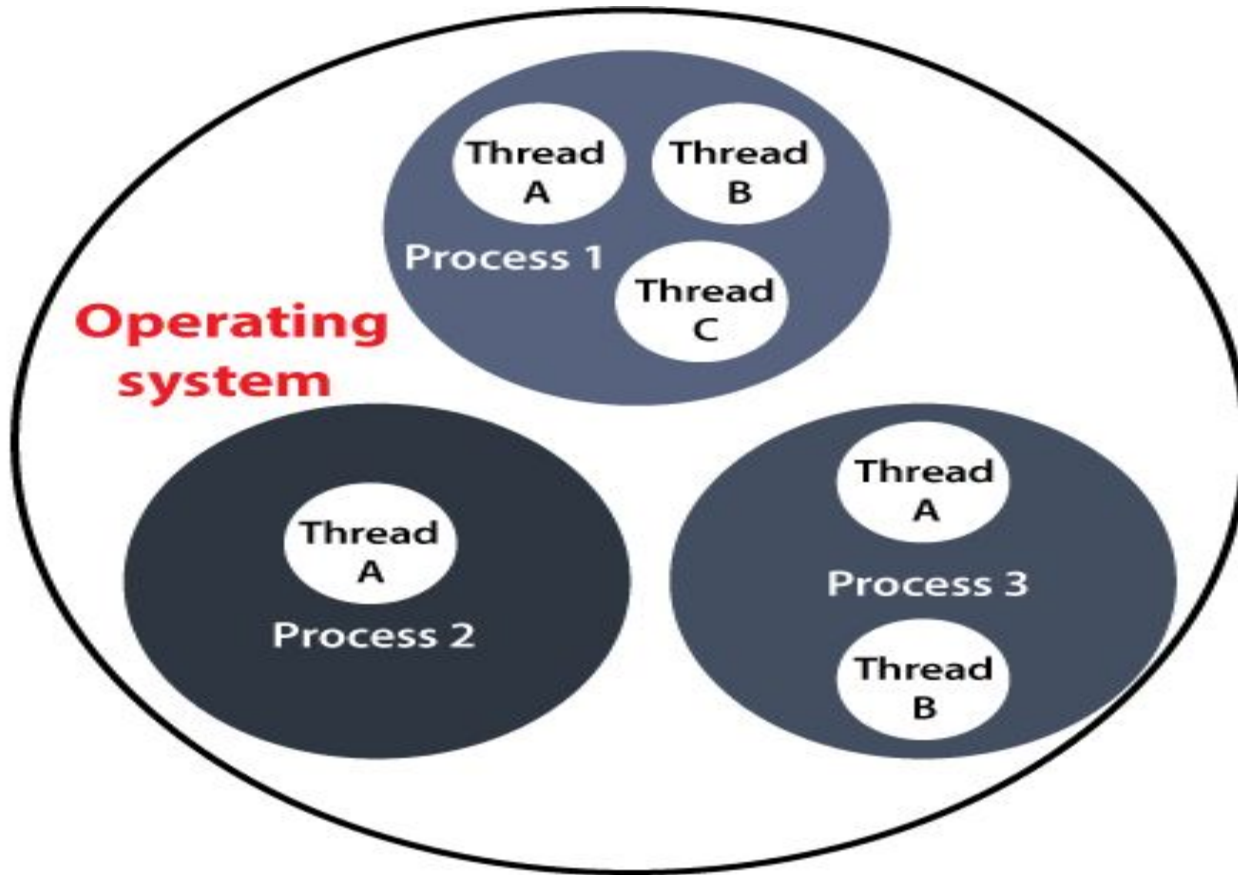
- In a thread-based multitasking environment, the thread is the smallest unit of dispatchable code.
- This means that a single program can perform two or more tasks simultaneously.
- For instance, a text editor can format text at the same time that it is printing, as long as these two actions are being performed by two separate threads.

INTRODUCTION TO MULTITHREADING

Thread-based multitasking :

- Threads, on the other hand, are lightweight.
- They share the same address space and cooperatively share the same heavyweight process.
- Interthread communication is inexpensive, and context switching from one thread to the next is low cost.
- Multithreading enables you to write very efficient programs that make maximum use of the CPU,

INTRODUCTION TO MULTITHREADING



JAVA THREAD MODEL

A thread is a:

- Facility to allow multiple activities within a single process
- A thread is a series of executed statements
- Each thread has its own program counter, stack and local variables

JAVA THREAD MODEL

STATES OF THREADS IN JAVA

1. New
2. Runnable
3. Running
4. Non-Runnable(Blocked)
5. Terminated

JAVA THREAD MODEL

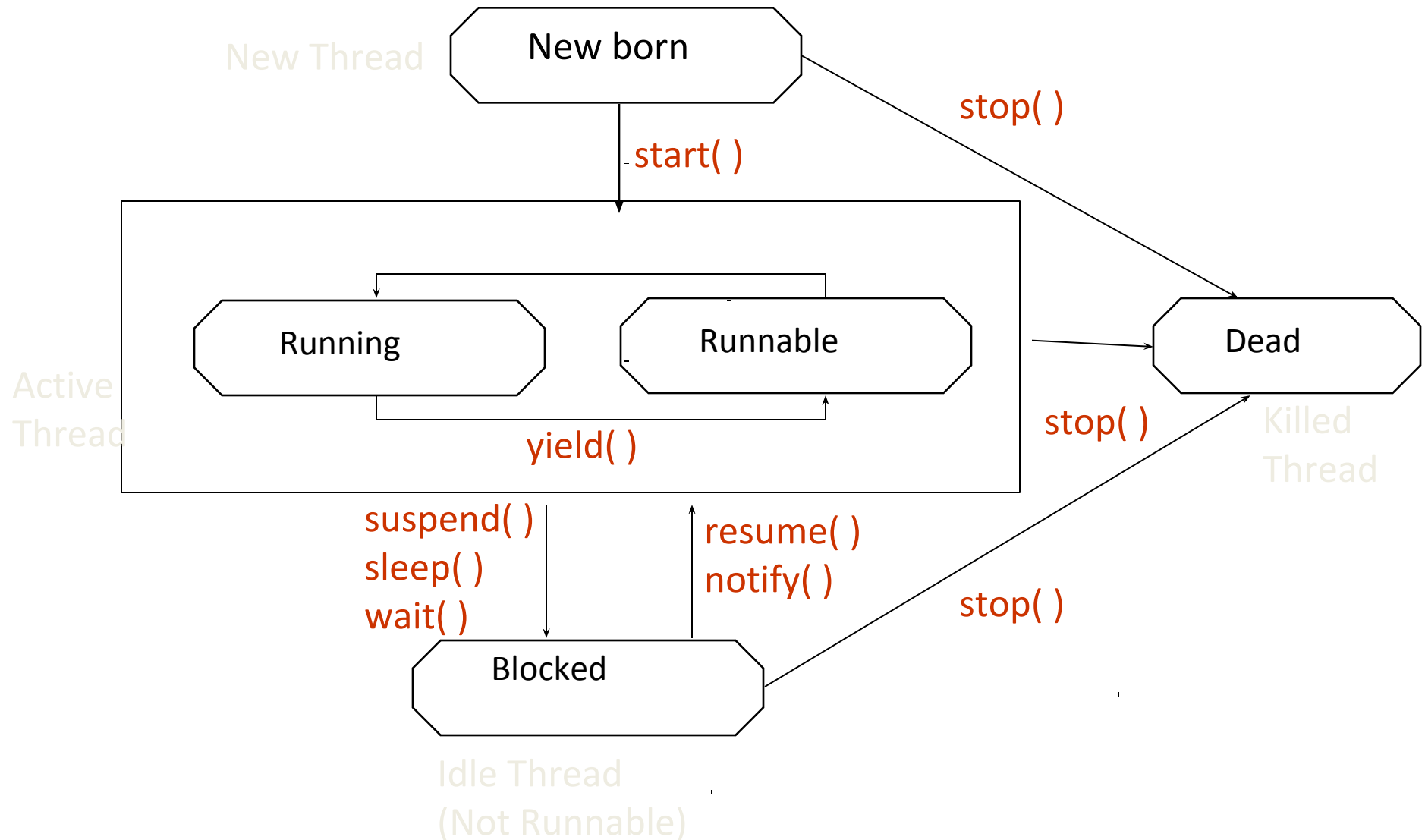


Fig:- State Transition diagram of a thread

THE MAIN THREAD

- When a Java program starts, one thread begins running immediately.
- This is usually called the main thread of the program, because it is the one that is executed when your program begins.
- The main thread is important for two reasons:
 - It is the thread from which other “child” threads will be spawned.
 - Often it must be the last thread to finish execution because it performs various shutdown actions.

THE MAIN THREAD

- Although the main thread is created automatically when your program is started, it can be controlled through a Thread object.
- To do so, you must obtain a reference to it by calling the method `currentThread( )`, which is a public static member of **Thread**.
- Its general form is:
`static Thread currentThread( )`
- This method returns a reference to the thread in which it is called.
- Once you have a reference to the main thread, we can control it just like any other thread.

Ramaiah Institute of Technology
(Autonomous Institute, Affiliated to VTU)
Department of Computer Science and Engineering

Course Name: Object Oriented Programming Course

Code: CS34

Credits: 3:0:0

Term: October – January 2023

Faculty:
Mamatha A

Ramaiah Institute of Technology
(Autonomous Institute, Affiliated to VTU)
Department of Computer Science and Engineering

UNIT IV

EXCEPTION HANDLING & MULTITHREADED PROGRAMMING

RECAP...

Multithreaded Programming:

- Introduction to MultiThreading
- Java Thread Model
- The Main Thread

AGENDA

Multithreaded Programming:

- Creating a Thread
- Creating Multiple Threads
- Using `isAlive()` and `Join()`
- Thread Priorities

CREATING A THREAD

- We can create a thread by instantiating an object of type **Thread**.
- Java defines two ways in which this can be accomplished:
 - By extend the **Thread** class.
 - By implementing the **Runnable** interface.

EXTENDING THREAD

- To create a thread is to create a new class that extends **Thread**
- Create an **instance of that class**.
- The extending class must override the **run()** method, which is the entry point for the new thread.
- A call to **start()** begins execution of the new thread.

IMPLEMENTING RUNNABLE

- To create a thread is to create a class that implements the Runnable interface.
- To implement Runnable, a class need only implement a single method called `run( )`, which is declared like this:
`public void run( )`
- Inside `run( )`, define the code that constitutes the new thread.
- A `run( )` can call other methods, use other classes, and declare variables, just like the main thread can.

IMPLEMENTING RUNNABLE

- The only difference is that `run( )` establishes the entry point for another, concurrent thread of execution within program. This thread will end when `run( )` returns.
- Creating a class that implements `Runnable`, we will instantiate an object of type `Thread` from within that class.
- `Thread` defines several constructors. The one that we will use is shown here:

`Thread(Runnable threadOb, String threadName)`

IMPLEMENTING RUNNABLE

- In this constructor, threadOb is an instance of a class that implements the Runnable interface.
- This defines where execution of the thread will begin.
- The name of the new thread is specified by threadName.
- After the new thread is created, it will not start running until we call its start() method, which is declared within Thread.

CREATING MULTIPLE THREADS

- Two threads: the main thread and one child thread.
- A Program can spawn as many threads as required

USING ISALIVE() AND JOIN()

- A Program wants the main thread to finish last.
- In the preceding examples, this is accomplished by calling `sleep( )` within `main( )`, with a long enough delay to ensure that all child threads terminate prior to the main thread.
- Two ways - To determine whether a thread has finished.

USING ISALIVE() AND JOIN()

- Two ways - To determine whether a thread has finished.
- First, we can call `isAlive( )` on the thread.
- This method is defined by Thread, and its general form is shown here:

`final boolean isAlive( )`

- The `isAlive( )` method returns **true** if the thread upon which it is called is still running. It returns false otherwise.

USING ISALIVE() AND JOIN()

□ Second one used to wait for a thread to finish is called join():

final void join() throws InterruptedException

- This method waits until the thread on which it is called terminates.
- Additional forms of join() allow us to specify a maximum amount of time that you want to wait for the specified thread to terminate.

Ramaiah Institute of Technology
(Autonomous Institute, Affiliated to VTU)
Department of Computer Science and Engineering

Course Name: Object Oriented Programming Course

Code: CS34

Credits: 3:0:0

Term: October – January 2023

Faculty:
Mamatha A

Ramaiah Institute of Technology
(Autonomous Institute, Affiliated to VTU)
Department of Computer Science and Engineering

UNIT IV

EXCEPTION HANDLING & MULTITHREADED PROGRAMMING

RECAP....

Multithreaded Programming:

- Creating a Thread
- Creating Multiple Threads
- Using `isAlive()` and `Join()`

AGENDA

Multithreaded Programming:

- Thread Priorities
- Synchronization
- Suspending, Resuming And Stopping Threads

THREAD PRIORITIES

- Thread priorities are used by the thread scheduler to decide when each thread should be allowed to run.
- Higher-priority threads get more CPU time than lower-priority threads.
- A higher-priority thread can also preempt a lower-priority one.
- For instance, when a lower-priority thread is running and a higher-priority thread resumes (from sleeping or waiting on I/O, for example), it will preempt the lower-priority thread.
- Threads of equal priority should get equal access to the CPU.

THREAD PRIORITIES

- To set a thread's priority, use the `setPriority( )` method, which is a member of `Thread`.
- This is its general form:
`final void setPriority(int level)`
- Here, `level` specifies the new priority setting for the calling thread.
- The value of `level` must be within the range `MIN_PRIORITY` and `MAX_PRIORITY`.
- Currently, these values are 1 and 10, respectively.

THREAD PRIORITIES

- To set a thread to default priority, specify `NORM_PRIORITY`, which is currently 5.
- These priorities are defined as static final variables within `Thread`.
- We can obtain the current priority setting by calling the `getPriority()` method of `Thread`, shown here:
`final int getPriority()`

SYNCHRONIZATION

- When two or more threads need access to a shared resource, they need some way to ensure that the resource will be used by only one thread at a time. The process by which this is achieved is called **synchronization**.
- Key to synchronization is the concept of the **monitor**.
- A **monitor** is an object that is used as a **mutually exclusive lock**.
- Only one thread can own a monitor at a given time.

SYNCHRONIZATION

- When a thread acquires a lock, it is said to have entered the monitor.
- All other threads attempting to enter the locked monitor will be suspended until the first thread exits the monitor.
- These other threads are said to be waiting for the monitor.
- We can synchronize the code by using the **synchronized** keyword in two ways:
 - Using Synchronized Methods
 - Using Synchronized Statement

SYNCHRONIZED METHODS

- All objects have their own implicit monitor associated with them.
- To enter an object's monitor, just call a method that has been modified with the synchronized keyword.
- While a thread is inside a synchronized method, all other threads that try to call it (or any other synchronized method) on the same instance have to wait.
- To exit the monitor and relinquish control of the object to the next waiting thread, the owner of the monitor must return from the synchronized method.
- [Program - Synch.java](#)

SYNCHRONIZED STATEMENT

- Creating synchronized methods within classes is an easy and effective means of achieving synchronization
- Imagine that we want to synchronize access to objects of a class that was not designed for multithreaded access. That is, the class does not use synchronized methods.
- Further, this class was not created by the programmer, but by a third party, and we do not have access to the source code. Thus, we can't add synchronized to the appropriate methods within the class.

SYNCHRONIZED STATEMENT

- How can access to an object of this class be synchronized?
The solution to this problem is quite easy: Put calls to the methods defined by this class inside a **synchronized block**.
- This is the general form of the synchronized statement:

```
synchronized(objRef)
{
    // statements to be synchronized
}
```

- Here, objRef is a reference to the object being synchronized.

- [Program – Synch1.java](#)

SUSPENDING, RESUMING, AND STOPPING THREADS

- The methods `suspend( )`, `resume( )`, and `stop( )`, which are methods defined by `Thread`, to pause, restart, and stop the execution of a thread.
- A thread must be designed so that the `run( )` method periodically checks to determine whether that thread should suspend, resume, or stop its own execution.
- Typically, this is accomplished by establishing a flag variable that indicates the execution state of the thread.
- As long as this flag is set to “running,” the `run( )` method must continue to let the thread execute.
- If this variable is set to “suspend,” the thread must pause.
- If it is set to “stop,” the thread must terminate.

Thank You...